AFA Platform - Geographic Information System

ISKOftware

Group members:
Justin Christian F. Chuah
Luis Antonio P. Sarmiento
Mansur Khalil B. Batistil

Client:
Office of Field Activities Diliman

Project blog:
https://miriitode.github.io/CS191-OFA-GIS/

CS 191 WFR1
Ligaya Leah Lara-Figueroa

Content Model

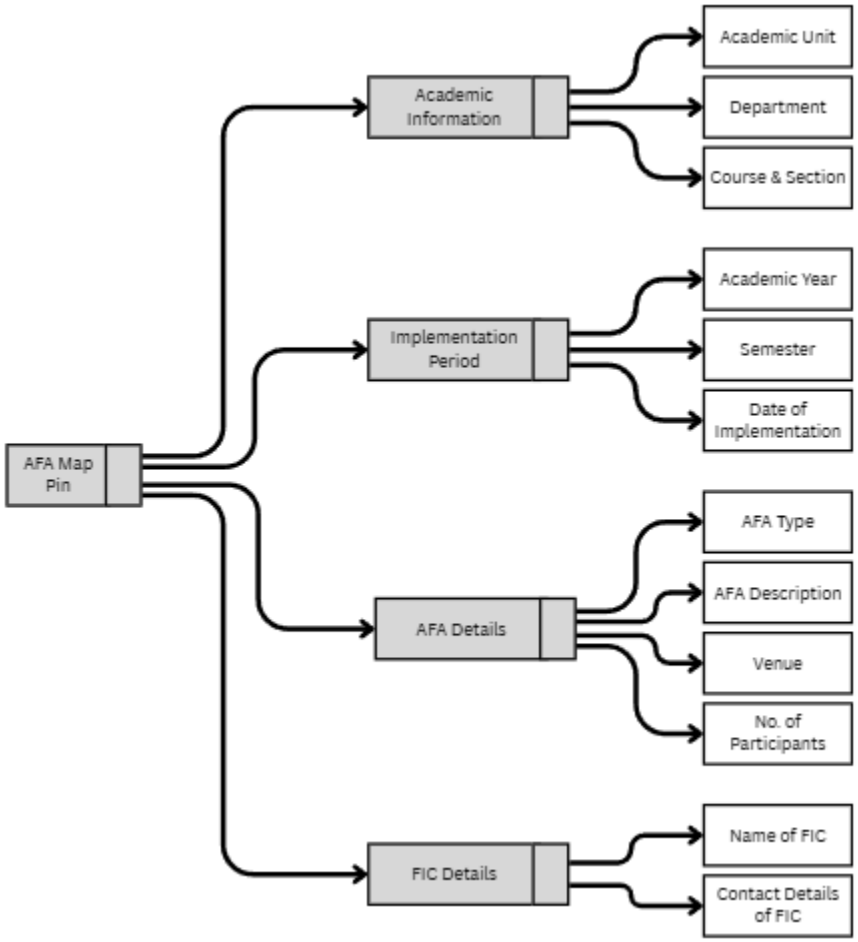


Figure 1: GIS Project Content Model for AFA Pins in map viewer

Interaction Model

Before the use cases planned in the final product are laid out, the actors and goals that will interact with the system must first be defined.

Actor	**Actor Type**	**Initiating Actor Goals**
Public user	Initiating	● View the GIS's interactive map ● Browse the AFAs displayed as pins in the map ● Click a pin to view the full details of the associated AFA ● Filter pins shown by a specific time period or range by setting a date input
Administrator	Initiating	● Add, edit, or delete records in the Google Sheets database ● Maintain accurate location coordinates and AFA descriptions ● Resolve invalid records ● Ensure changes in the Google Sheets database are properly reflected on the GIS's interactive map
Google Sheets database	Participating	N/A

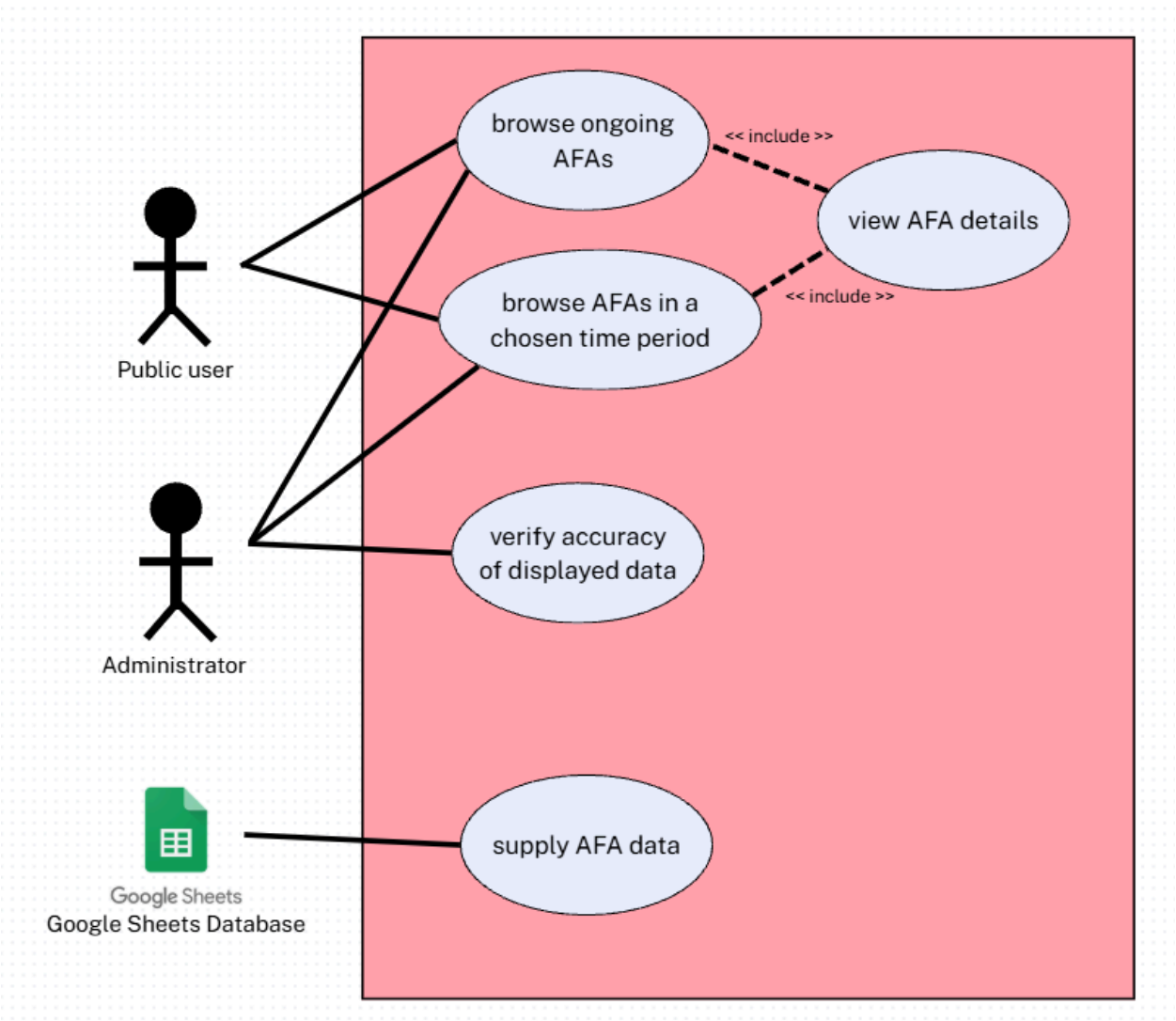


Figure 2: GIS Project Use Case Diagram

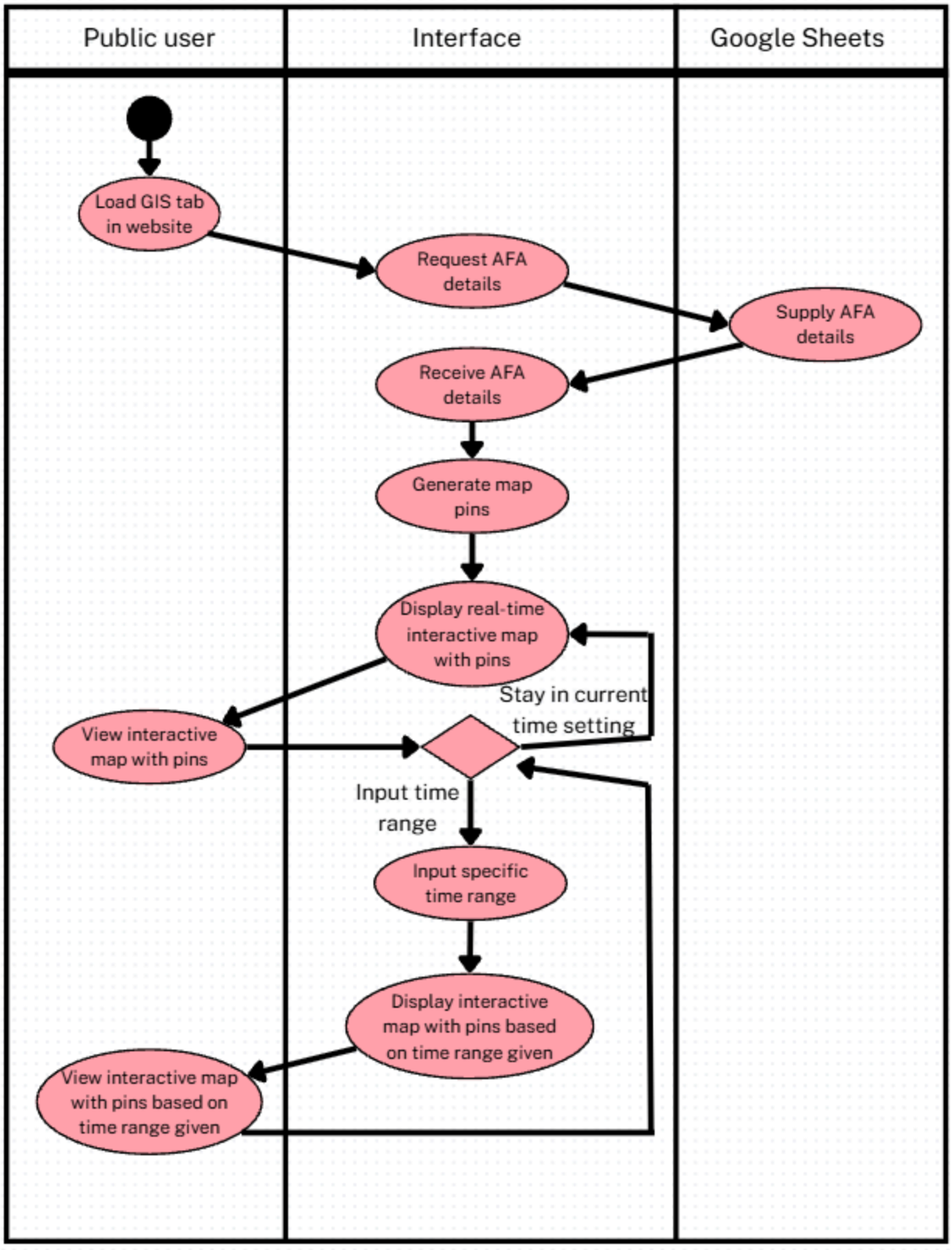


Figure 3: GIS Project Swimlane Diagram for viewing AFA information in GIS map viewer

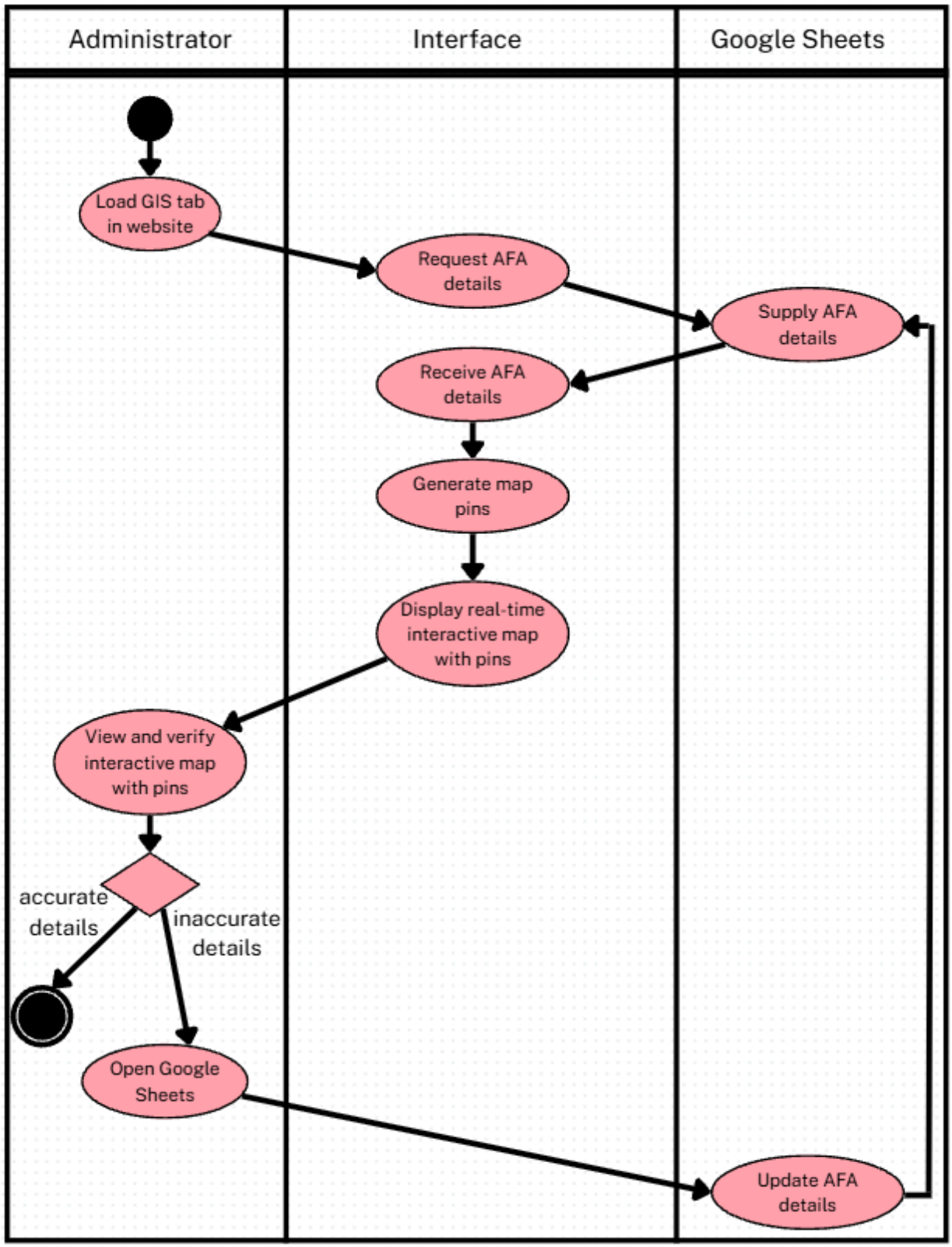


Figure 4: GIS Project Swimlane Diagram for updating AFA information in GIS map viewer

Configuration Model

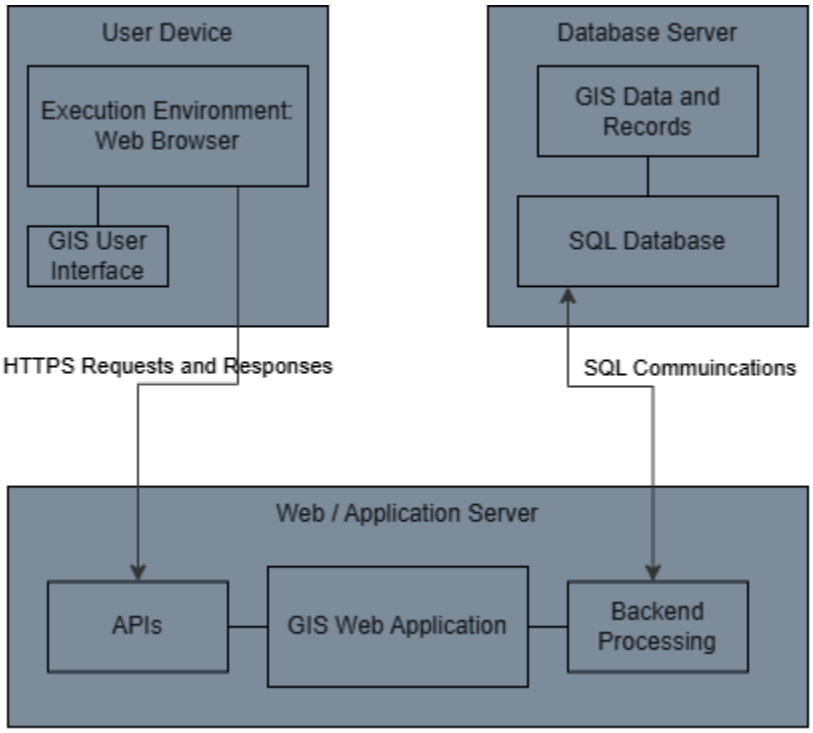


Figure 5: GIS Project UML Deployment Diagram

The ideal allocation of modules are as follows in the model shown. The User Interface of the webApp and the API calls between the user and the execution environment of the webApp itself reside under the user device via the web browser. This is where user requests are generated, such as requesting for the website itself, obtaining all the components and relevant data in order for the website to function. The actual GIS webApp and the APIs responsible for communicating to and from the user is handled under the Website / Application Server. It also handles any requests incoming from users interacting with the website. Lastly, the backend processing, such as obtaining the relevant AFA information from the database is also handled by the Website / Application server. The Website / Application server then handles requests made by the user to the Database Server, where it will then query the database and return the appropriate responses relative to the queries.

Ideally this model would require the use of at least 3 machines: The user's device, the Website / Application Server, and the Database Server. However, it may be possible to combine two of the three together such as components from the User Device and the Website / Application Server (by having the server send all the data required to run the GIS application to the device and have it run there locally), or the Website / Application server and the Database Server (by containing the Database Management System within the same machine as the Website / Application Server).

GIS Architecture and Design

1. **Architecture Design**

Considering the nature of the GIS application, it would be using the Network Structure. The GIS application displays all relevant AFA pins in one place, and the user can be able to select any pin and show its corresponding information, regardless of where the user is in the website. If the user is viewing an AFA, they are able to view any other AFA, as shown in the diagram below, similar to the figure presented in Pressman's "Software Engineering: A Practitioner's Approach" (Chapter 13.7.1, Figure 13.7).:

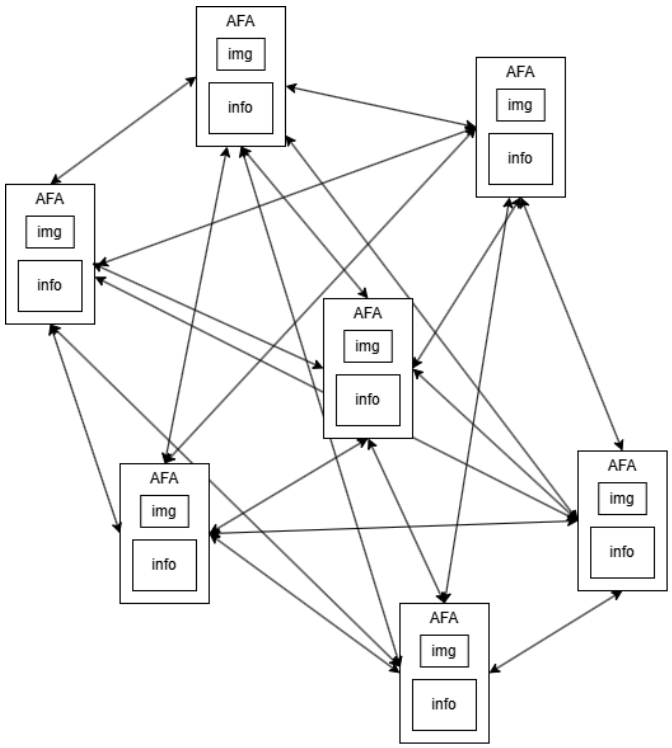


Figure 6: Network Structure for Architecture Design of the GIS WebApp

2. **WebApp Architecture**
   The diagram below shows how all the components of the WebApp interface with one another:

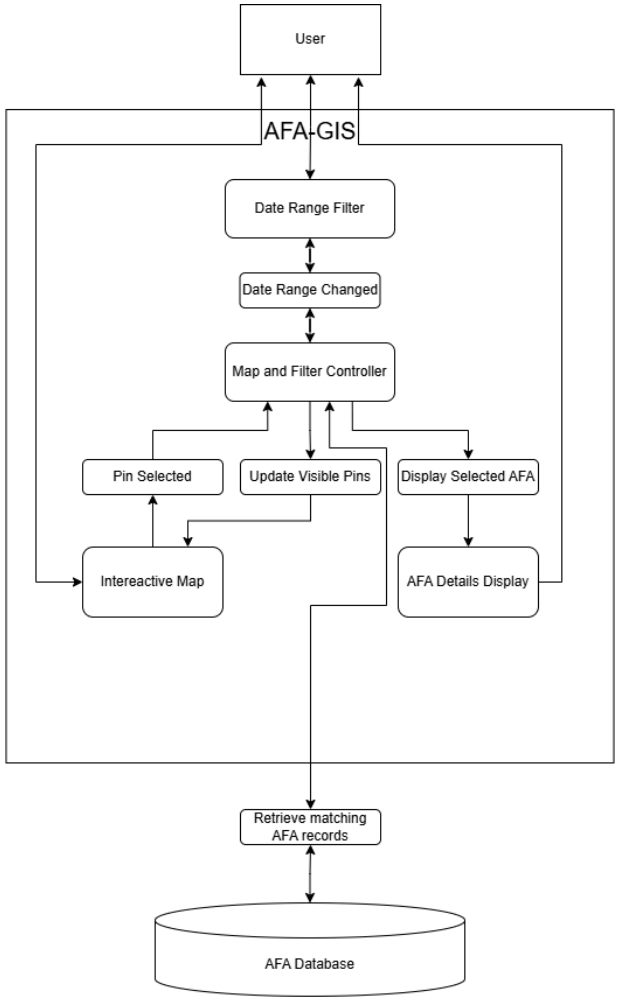


Figure 7: WebApp Architecture of the GIS WebApp

These designs address the technical attributes of the “quality requirement tree” from Pressman (Chapter 13.1, Figure 13.1) by the following:

- Usability
    - The website contains a simple how to use box and is easily accessible in case the user needs them again. The site’s controls are simple, focusing on moving the map, selecting the pins, and selecting the date range.
- Functionality
    - The website contains core functionality such as the map viewer, pin selections that show the selected AFA, and the date filter, which updates the pins of AFAs in the map.
- Reliability
    - Each module has checks in order for the application to properly and correctly perform the actions defined in their modules, and have certain

measures in place to address any possible errors that could manifest in the application.

- Efficiency
    - With the use of open-source map viewers, and the system infrastructure in communication between the server and the database, this allows for certainty that the user interactions are quick and smooth to deliver the relevant information to the user.
- Maintainability
    - Since each component of the WebApp is to be modular, it would allow for the website to be easily maintained by isolating any issue to a specific module. It also allows for easily being able to add more features to the website in the future when the clients request for it.

Content Design

i. Class Diagrams

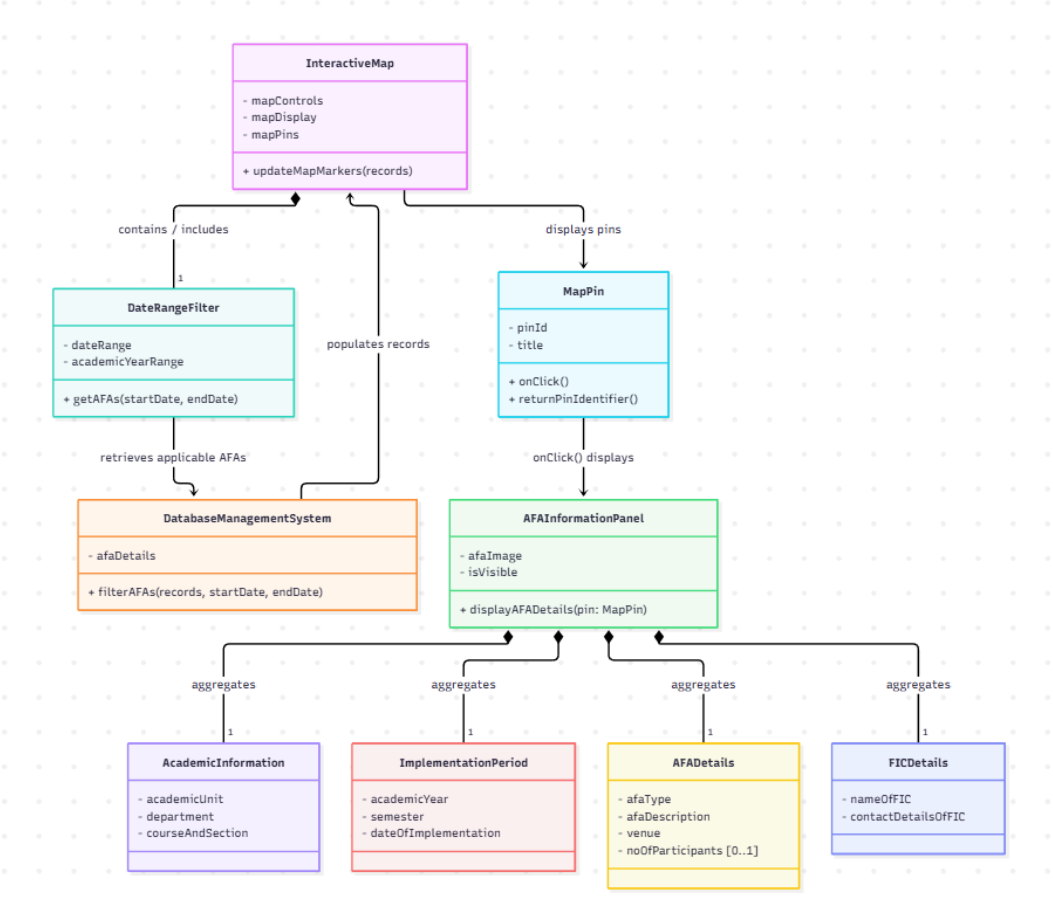


Figure 8: GIS WebApp Class Diagram

ii. Entity-Relationship Diagram

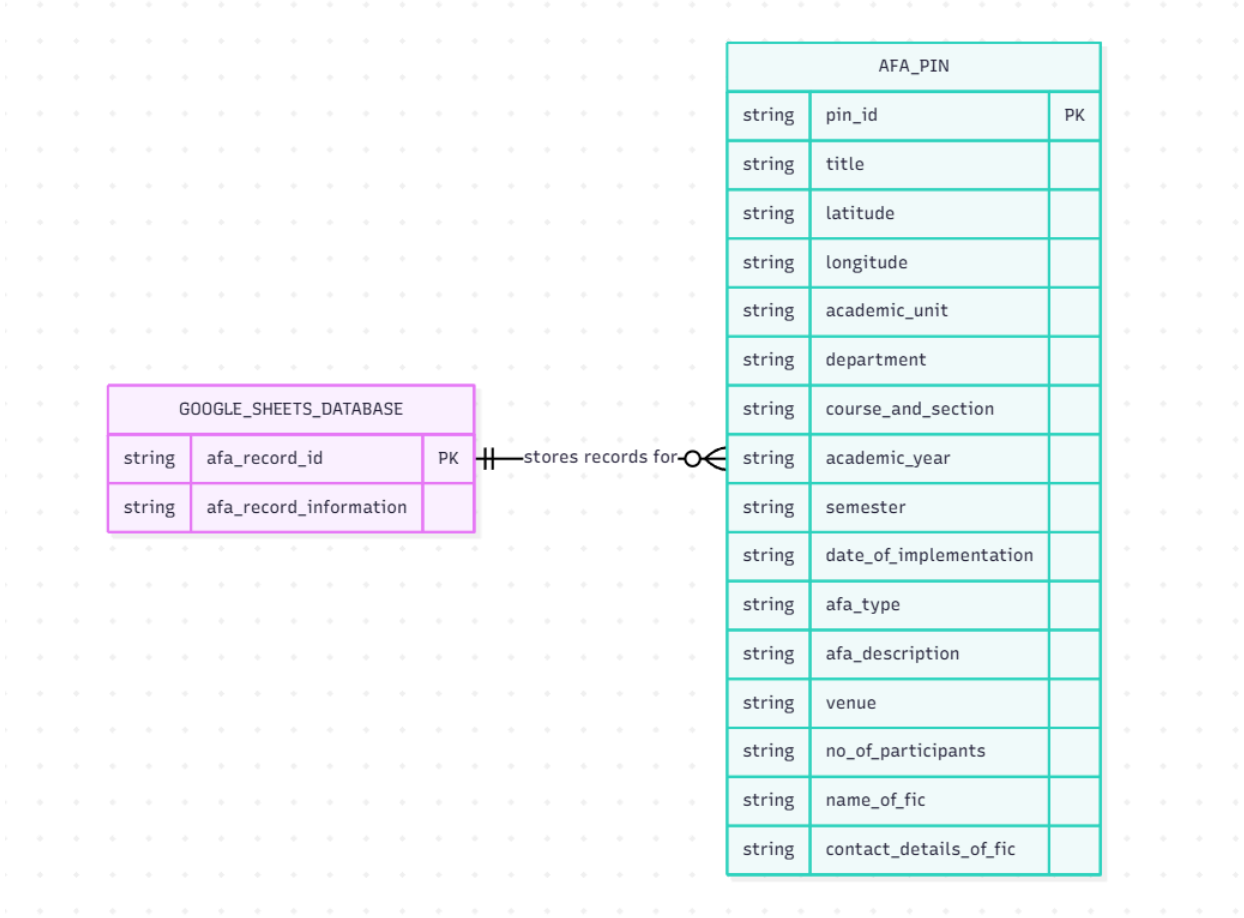


Figure 9: GIS Entity Relationship Diagram

iii. Data Flow Diagrams

DFD Level 0

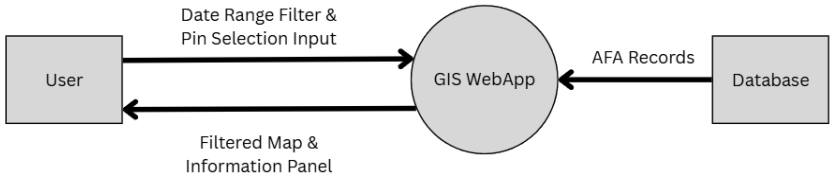


Figure 10: Level 0 Data Flow Diagram of the GIS WebApp

DFD Level 1

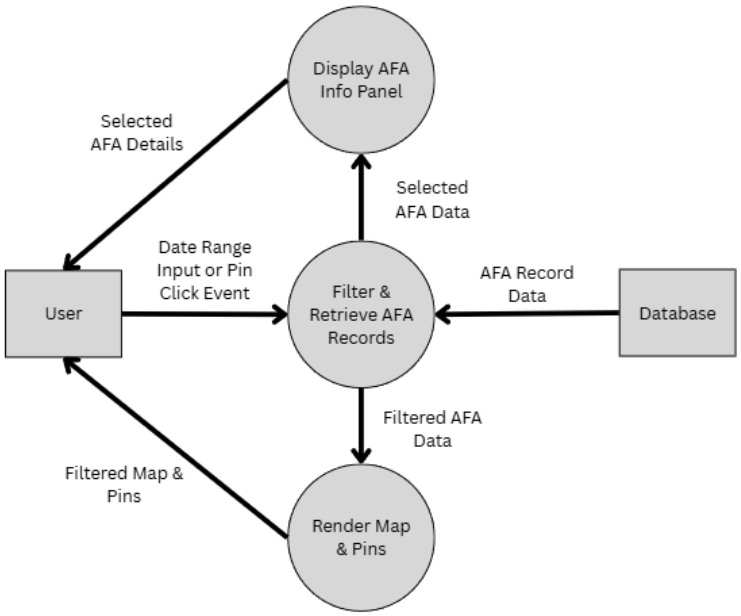


Figure 11: Level 1 Data Flow Diagram of the GIS WebApp

DFD Level 2

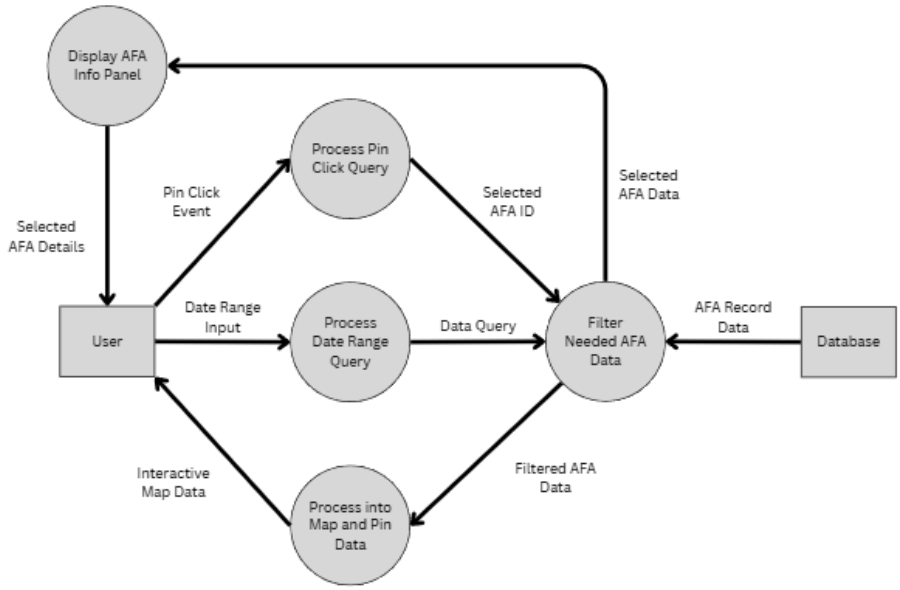


Figure 12: Level 2 Data Flow Diagram of the GIS WebApp

Component-Level Design

The table below shows all the necessary components for the GIS Application:

Component	Role	Input / Output
Interactive Map	Displays the map, map pins, and handles pin selection logic from the user	AFA Locations / Map Markers
Date Range Filter	Handles user interaction and logic for specifying date ranges	Start and End dates OR Academic Year and Semester / Filter Criteria for Database Query
Database Management System	Handles sending queries, and retrieving relevant AFA data from the AFA Database	Map Marker AND Filter Criteria for Database Query / Relevant AFA information
Details Display	Shows the information of the selected AFA	Selected AFA record / Detailed Side Panel

For each of the components, they have the following interface representations:

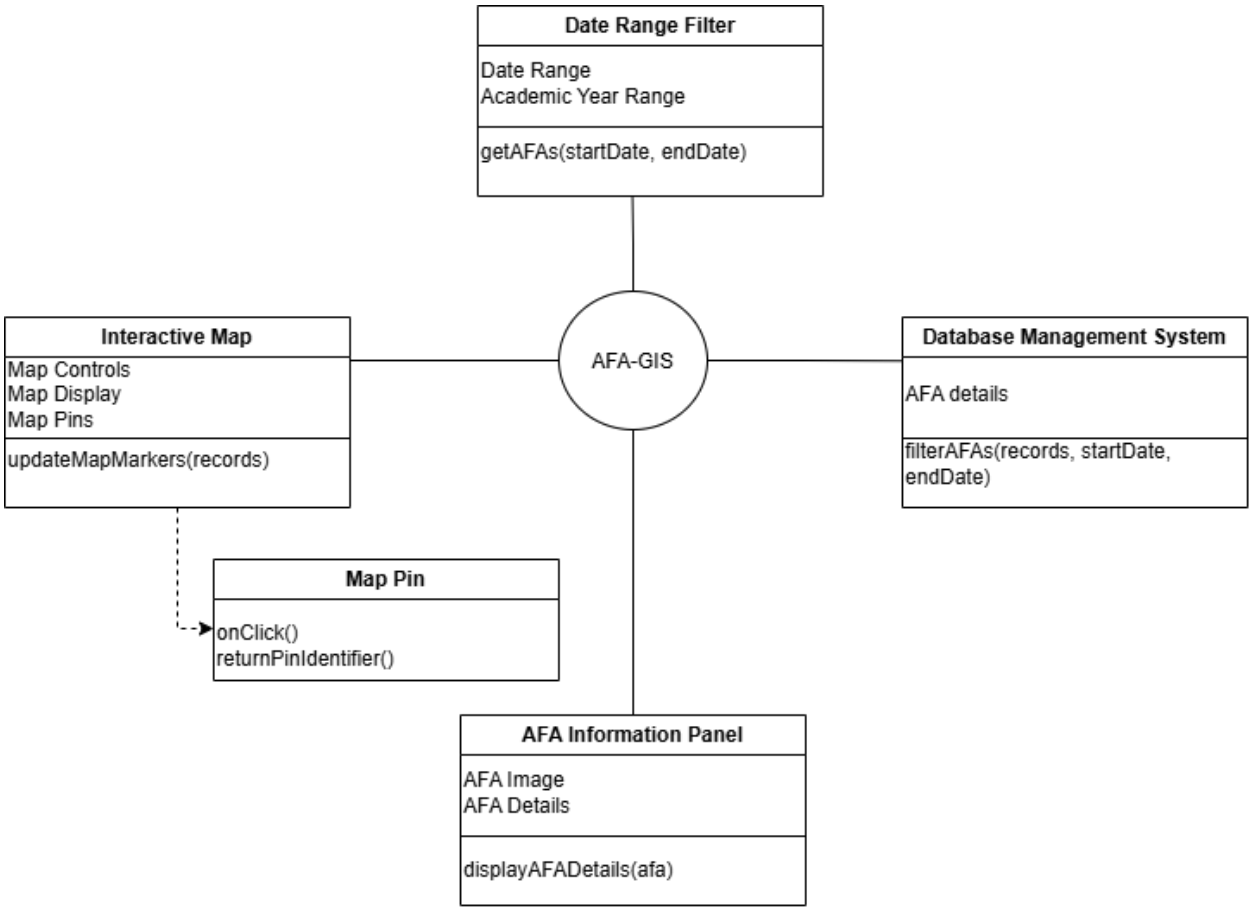


Figure 13: Interface Representation of the WebApp

The table below shows the dependencies of each of the components:

Interactive map -> AFA Details Display
Interactive map -> Date Range Filter
Date Range Filter -> Database Management System
Database Management System -> Interactive Map

Given the simplicity of the application's purpose, the logic mainly revolves around date-range filtering and handling the map pin selection logic to correctly map to the database query to return the correct information.

The following diagrams below illustrate the algorithm for the date-range filtering, and selecting a map pin:

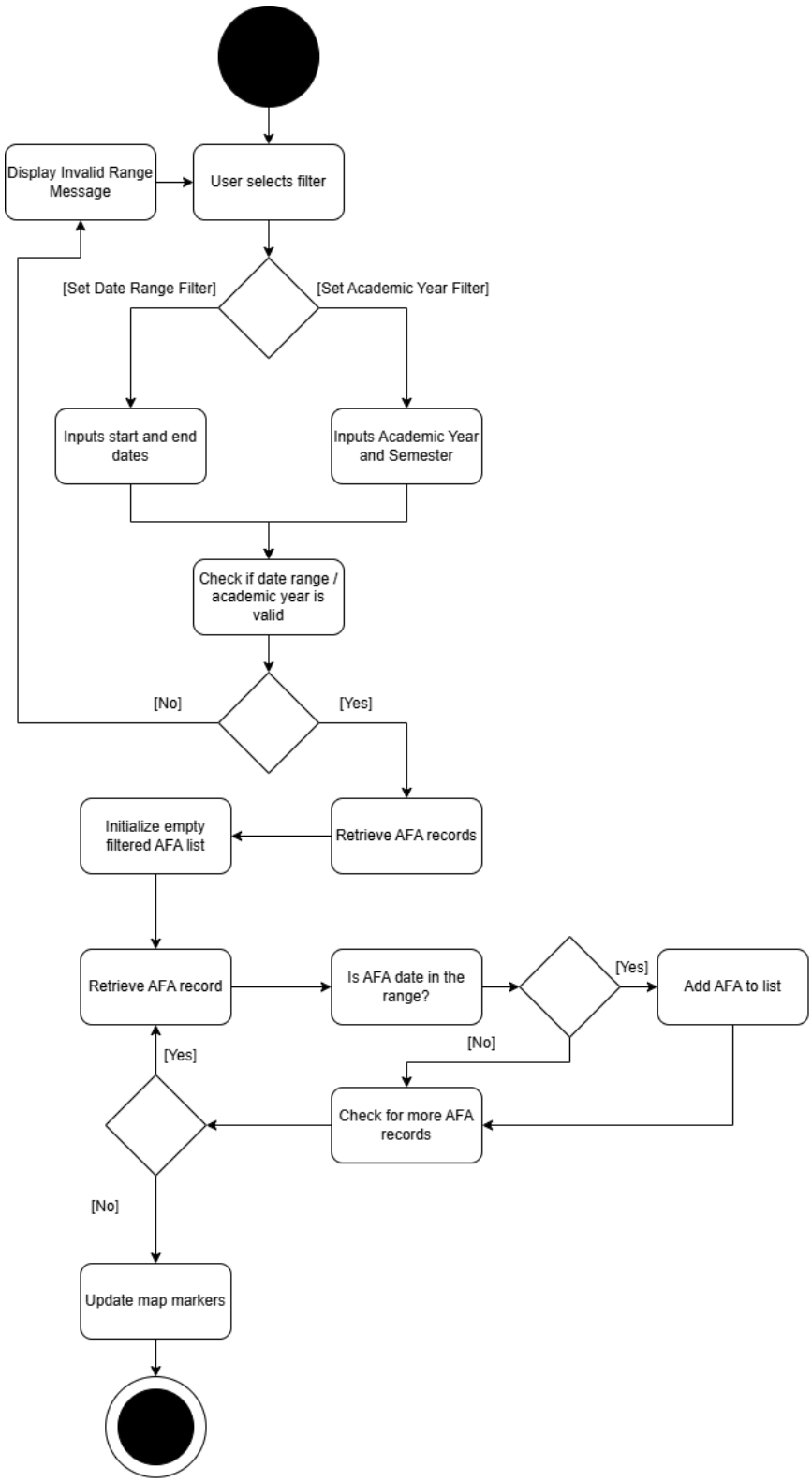


Figure 14: UML Activity diagram for Date Range Filter Algorithm

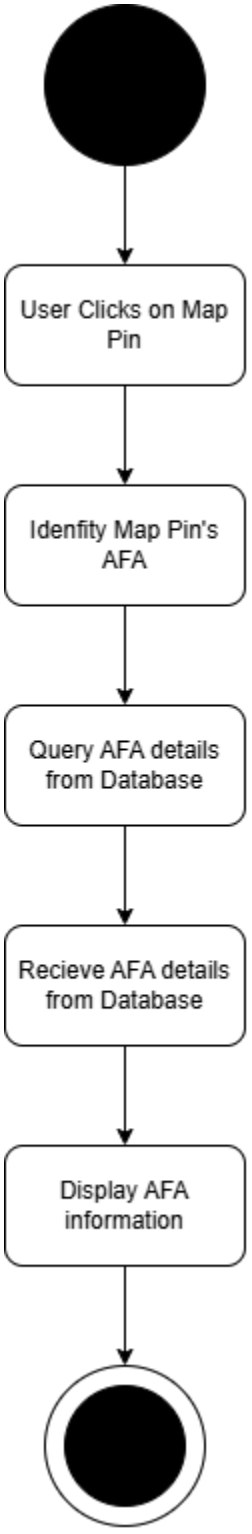


Figure 15: UML Activity diagram for Map Pin Selection Algorithm

Given how simple the data is for displaying information of AFAs, the only data structure relevant to the application is for the necessary information to display for a given AFA. This record could be easily stored as data structures that are suitable for data transmission from the database to the user device, such as JSON. The following is a sample structure of the AFA, not factoring any extra information found from the different classifications of AFAs, but can be added into this data structure easily by adding more fields:

```JSON
{
	"id": 1,
	"name": "Field Activity A",
	"date": "2023-10-10",
	"afa_type": "on-campus",
	"location": "University of the Philippines Diliman".
	"latitude": 14.6539,
	"longitude": 121.0685,
	"description": "Lorem ipsum dolor sit amet."
}
```

The application would be using COTS. Mainly one for the map viewer, which will be Leaflet (https://leafletjs.com/). This allows for a simple map viewer to be ready, and for the development to focus on creating the other logic to display and filter AFAs.

Interface Design

With the use cases for utilizing the GIS having already been previously defined, an interface can now be designed to achieve the objectives of these use cases. A layout of user objectives into interface actions are shown in the figure below.

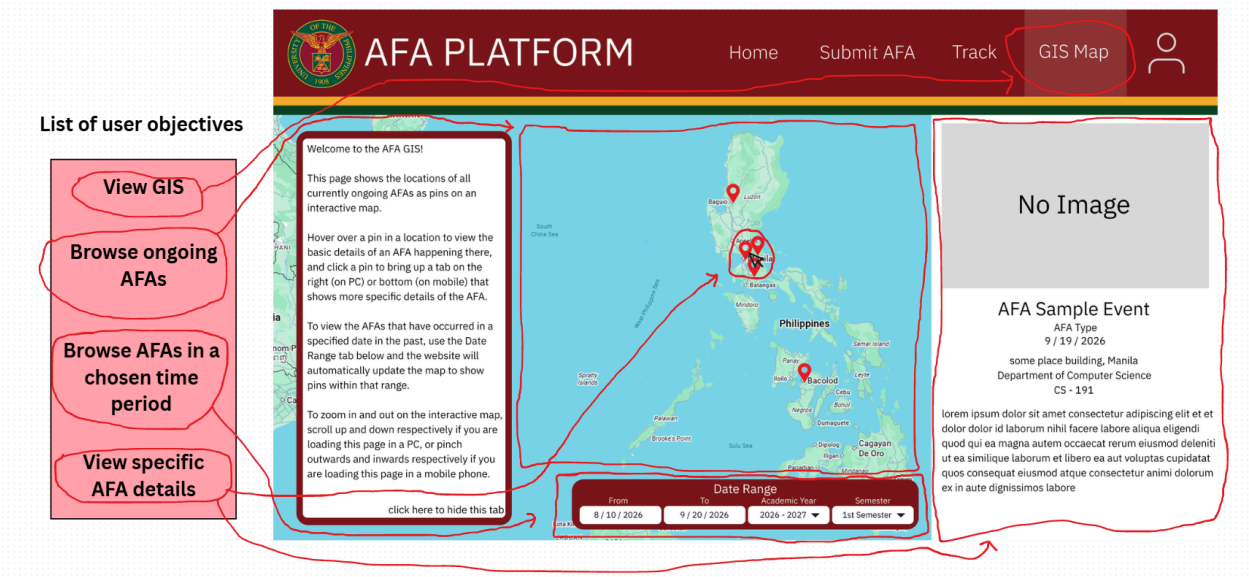


Figure 16: Mapping GIS Objectives into Interface Actions

Aesthetic Design

In the templates shown below for the aesthetic design of the GIS application, besides the map, the sidebars for the tutorial, date range selection, and AFA detailed information consist of a color scheme of maroon, black, and white. This design was chosen to signify and compliment the maroon color associated with the University of the Philippines Diliman as a whole as well as the predominant color in the top bar of the website. Important text to be read or edited are in a black color enclosed in white backgrounds for ease of readability. Finally, the tutorial sidebar can be hidden by clicking on the text of its bottom right portion to reduce clutter and show more of the interactive map, and can later be shown again by clicking the white question mark in the small maroon square that replaces the tutorial sidebar when it is hidden.

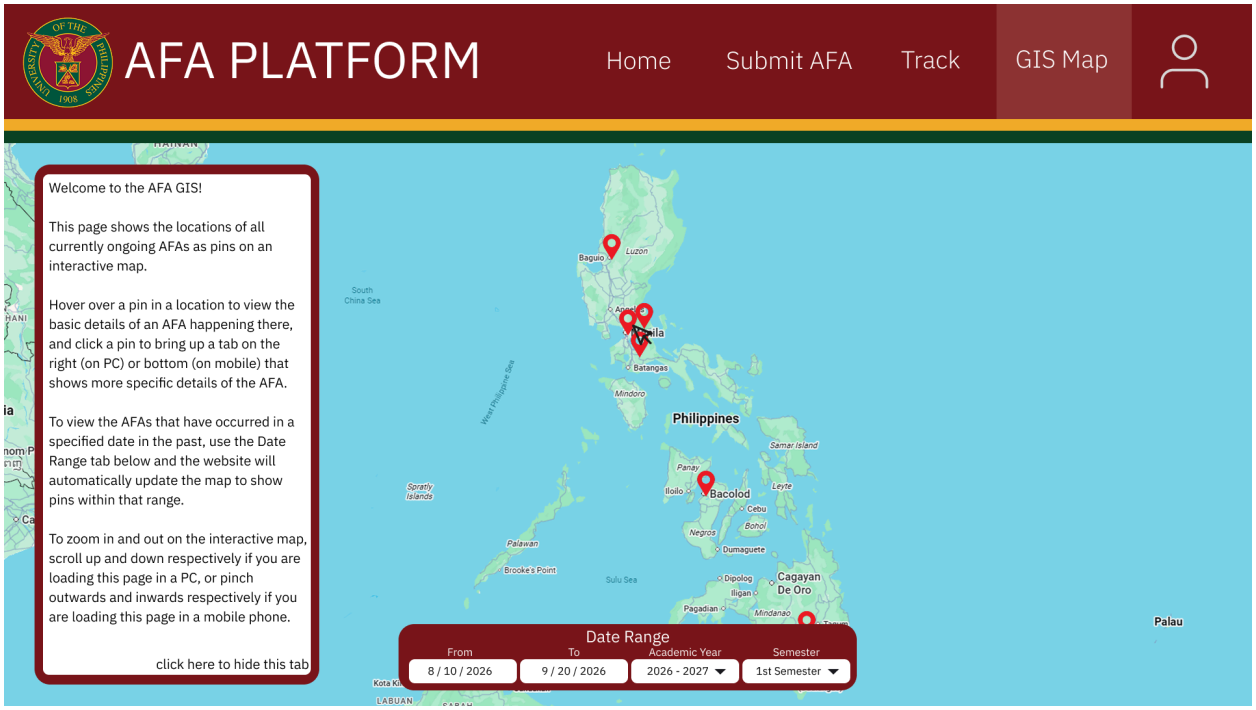


Figure 17: AFA - GIS Page Design Template on initial load on PC

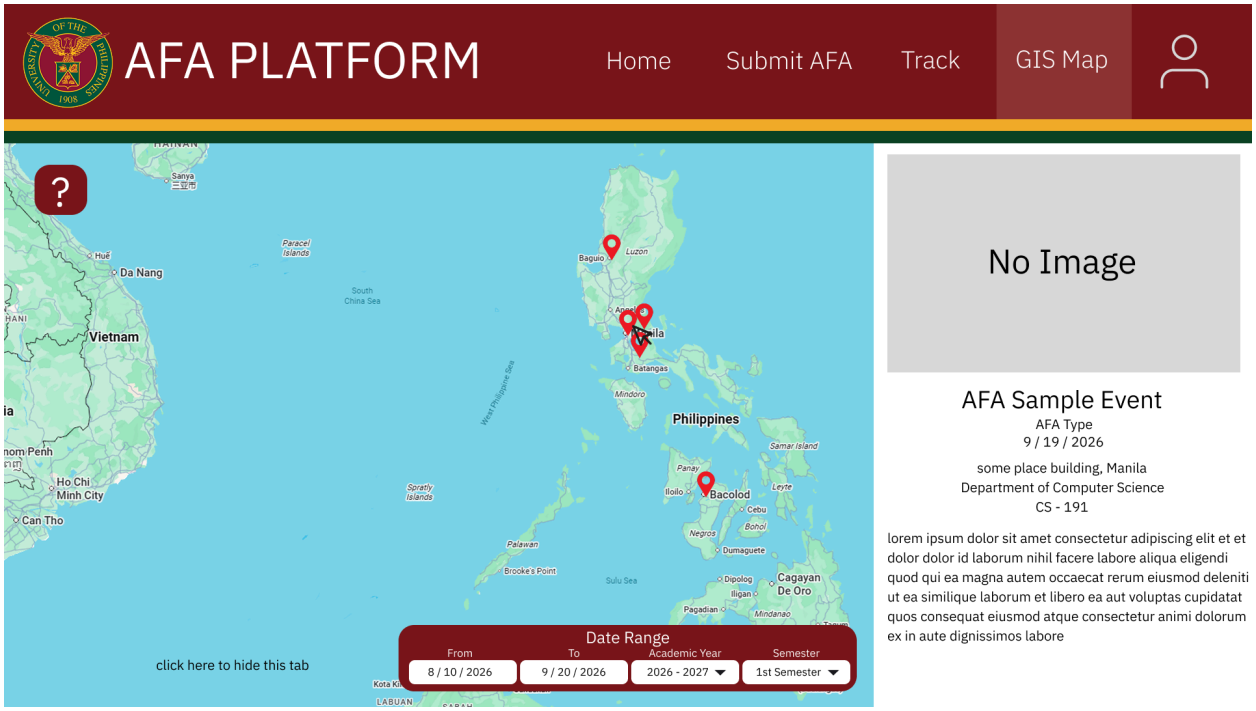


Figure 18: AFA - GIS Page Design Template showing detailed information and with tutorial sidebar hidden on PC

As shown in the figure above, the format of the detailed information from top to bottom is as follows: Image associated with the AFA; AFA title; AFA type; date; location; department; subject; description.

Navigation Design

Because everything needed to achieve the user objectives of the GIS is within the GIS page, navigation links are not included in the navigation design of the application. Using the previously shown page design templates for the application, once the GIS is loaded by clicking on the “GIS Map” icon in the top bar, there is no need to click the mouse to show the basic information of AFAs that are happening in real-time because only hovering the mouse over the desired AFA pins is necessary. If a user wants to see detailed information about an ongoing AFA, one mouse click is needed to click on the specified AFA and bring up the detailed information sidebar.

If a user wants to browse the AFAs that happened within a specified time period, 4 mouse clicks are necessary in order to fill in the 4 necessary tabs in the Date Range sidebar, and once the AFAs of that time period are shown, only 1 more mouse click is necessary to bring up the detailed information of an AFA in it, thus bringing the number of mouse clicks needed to accomplish the longest task (viewing detailed information on an AFA that happened within a certain time period) to 5.

Clerical data entry is required to view AFAs in a certain time period because data needs to be input in the Date Range sidebar to display these, but the rest of navigation goes to user-interface navigation through reading information, hovering over pins, and zooming into the interactive map.